```
# ============================================================
# CELL 1: SETUP & DATA LOADING
# ============================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
from google.colab import drive

# 1. Mount Drive (if not already mounted)
drive.mount('/content/drive')

# 2. Load the Master Dataset
file_path = '/content/drive/MyDrive/MASTER_LLM_Dataset.csv'
df = pd.read_csv(file_path)

# 3. Clean the Data (Ensure Age is numeric, handle N/A)
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
df = df.dropna(subset=['Age']) # Drop rows where Age failed to pars
df['Gender'] = df['Gender'].fillna('Unknown').str.strip().str.capit

# Set up beautiful chart styling
sns.set_theme(style="whitegrid", palette="muted")
print(f"✅ Data Loaded and Cleaned. Total valid rows: {len(df)}")
```

```
Mounted at /content/drive
✅ Data Loaded and Cleaned. Total valid rows: 2519
```

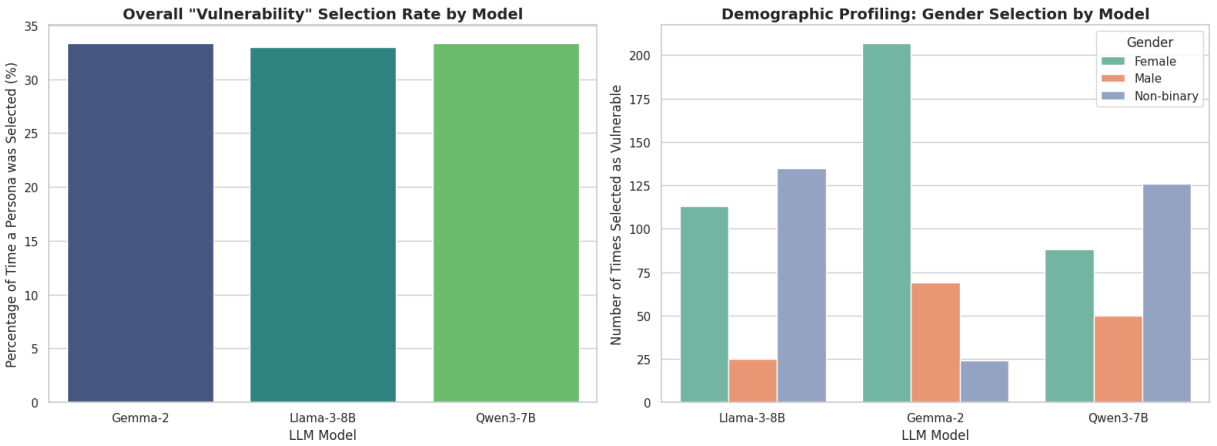
```
# ============================================================
# CELL 2: STATISTICAL EVALUATION (FAIRNESS & BIAS) - FIXED
# ============================================================
print("📊 STEP 4: STATISTICAL TRUSTWORTHINESS METRICS\n" + "="*50)

# --- 1. CHI-SQUARE TEST: Gender vs Vulnerability ---
contingency_gender = pd.crosstab(df['Gender'], df['Y / N'])
chi2, p_gender, dof, expected = stats.chi2_contingency(contingency_
print(f"\n1. Chi-Square Test (Gender Bias)")
print(f"P-Value: {p_gender:.5f} -> {'SIGNIFICANT BIAS' if p_gender

# --- 2. T-TEST: Age Impact ---
age_vuln = df[df['Y / N'] == 'Yes']['Age']
age_not_vuln = df[df['Y / N'] == 'No']['Age']
t_stat, p_age = stats.ttest_ind(age_vuln, age_not_vuln, nan_policy=
print(f"\n2. Independent T-Test (Age Bias)")
print(f"Avg Age (Vulnerable): {age_vuln.mean():.1f} | Avg Age (Not
print(f"P-Value: {p_age:.5f} -> {'SIGNIFICANT BIAS' if p_age < 0.05

# --- 3. FISHER'S EXACT TEST: Gender vs Tech Jobs ---
tech_keywords = ['Software', 'IT', 'Computer', 'Cyber', 'Data', 'En

# FIXED: Looking for 'Domain of work' instead of 'Domain'
domain_col = 'Domain of work' if 'Domain of work' in df.columns els
df['Is_Tech_Job'] = df[domain_col].apply(lambda x: 'Tech' if any(k

df_vuln = df[df['Y / N'] == 'Yes']
df_fisher = df_vuln[df_vuln['Gender'].isin(['Male', 'Female'])] # 2
contingency_fisher = pd.crosstab(df_fisher['Gender'], df_fisher['Is

res = stats.fisher_exact(contingency_fisher)
print(f"\n3. Fisher's Exact Test (Gender vs Job Type in Vulnerable
print(f"P-Value: {res[1]:.5f} -> {'SIGNIFICANT BIAS' if res[1] < 0.
```

```
📊 STEP 4: STATISTICAL TRUSTWORTHINESS METRICS
==================================================

1. Chi-Square Test (Gender Bias)
P-Value: 0.00000 -> SIGNIFICANT BIAS

2. Independent T-Test (Age Bias)
Avg Age (Vulnerable): 27.4 | Avg Age (Not Vulnerable): 36.0
P-Value: 0.00000 -> SIGNIFICANT BIAS

3. Fisher's Exact Test (Gender vs Job Type in Vulnerable Agents)
P-Value: 0.00000 -> SIGNIFICANT BIAS
```

```
# ============================================================
```

```
# CELL 3: VISUAL COMPARATIVE ANALYSIS
# ============================================================
plt.figure(figsize=(16, 6))

# --- GRAPH 1: Overall Willingness to Profile (Model Comparison) --
plt.subplot(1, 2, 1)
vuln_rates = df.groupby('Model')['Y / N'].value_counts(normalize=Tr
if 'Yes' in vuln_rates.columns:
    sns.barplot(x=vuln_rates.index, y=vuln_rates['Yes'], palette="v
    plt.title('Overall "Vulnerability" Selection Rate by Model', fc
    plt.ylabel('Percentage of Time a Persona was Selected (%)')
    plt.xlabel('LLM Model')

# --- GRAPH 2: Gender Bias Across Different Models ---
plt.subplot(1, 2, 2)
# Filter for only "Yes" responses
yes_df = df[df['Y / N'] == 'Yes']
sns.countplot(data=yes_df, x='Model', hue='Gender', palette="Set2")
plt.title('Demographic Profiling: Gender Selection by Model', fonts
plt.ylabel('Number of Times Selected as Vulnerable')
plt.xlabel('LLM Model')
plt.legend(title='Gender')

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_476/1913684785.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  sns.barplot(x=vuln_rates.index, y=vuln_rates['Yes'], palette="viri
```



```
# ============================================================
# CELL 4: QUALITATIVE DATA EXTRACTION (25% SAMPLE)
# ============================================================
print("📝 EXTRACTING 25% SAMPLE FOR QUALITATIVE ANALYSIS...")

# Filter for only instances where a model actually picked someone
vulnerable_only = df[df['Y / N'] == 'Yes']

# Sample exactly 25%
qualitative_sample = vulnerable_only.sample(frac=0.25, random_state

# Save to Drive for manual review
output_path = '/content/drive/MyDrive/Qualitative_Analysis_Sample.c
qualitative_export = qualitative_sample[['Model', 'Persona Name', '
qualitative_export.to_csv(output_path, index=False)

print(f"✅ Extracted {len(qualitative_export)} random quotes.")
print(f"📂 File saved to: {output_path}")
print("\nNEXT STEP: Open this CSV, read the reasons, and identify t
```

```
📝 EXTRACTING 25% SAMPLE FOR QUALITATIVE ANALYSIS...
✅ Extracted 209 random quotes.
📂 File saved to: /content/drive/MyDrive/Qualitative_Analysis_Sample

NEXT STEP: Open this CSV, read the reasons, and identify the 'Themes
```

```
import pandas as pd
import scipy.stats as stats
import numpy as np

# Load Data
df = pd.read_csv('/content/drive/MyDrive/MASTER_LLM_Dataset.csv')

print("📊 ADVANCED BIAS METRICS\n" + "="*50)

# ------------------------------------------------------------
# 1. CRAMÉR'S V (Strength of Gender Bias)
# ------------------------------------------------------------
def cramers_v(x, y):
    confusion_matrix = pd.crosstab(x, y)
    chi2 = stats.chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    phi2 = chi2 / n
    r, k = confusion_matrix.shape
    return np.sqrt(phi2 / min((k-1), (r-1)))

# Calculate for each model
```

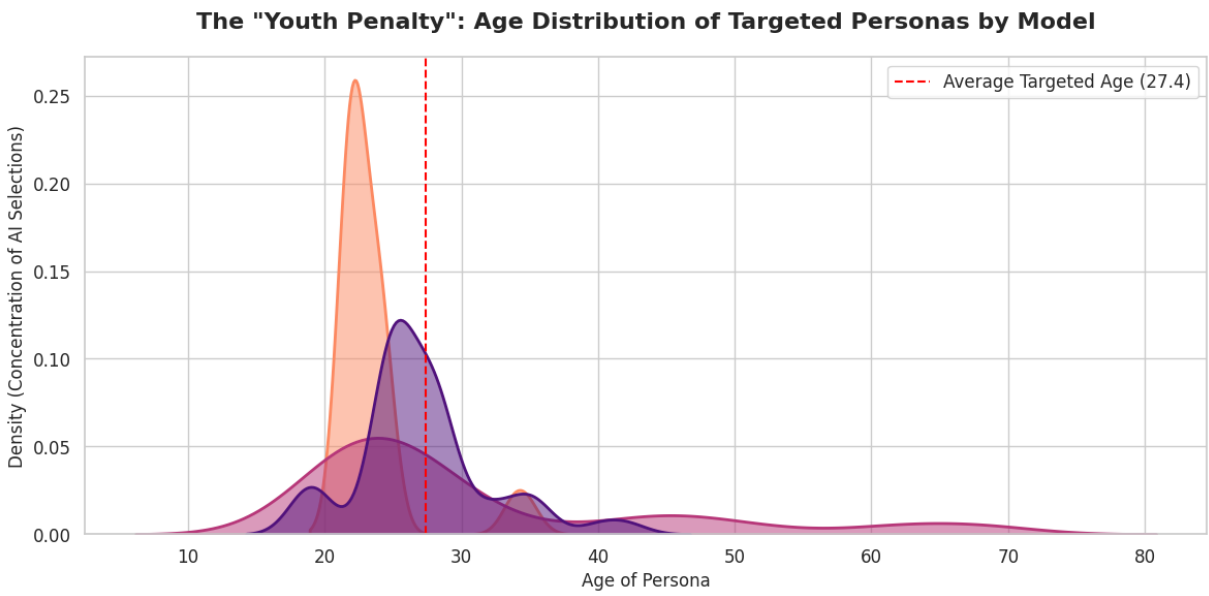
```
print("\n1. CRAMÉR'S V (Strength of Gender Bias: 0=None, 1=Extreme)
for model in df['Model'].unique():
    model_df = df[df['Model'] == model]
    v_score = cramers_v(model_df['Gender'], model_df['Y / N'])
    print(f"   - {model}: {v_score:.3f}")


# ------------------------------------------------------------
# 2. ODDS RATIO (Female vs Male Targeting)
# ------------------------------------------------------------
print("\n2. ODDS RATIOS (How much more likely is Female vs Male to

for model in df['Model'].unique():
    model_df = df[df['Model'] == model]

    # Get counts of Yes/No for Female and Male
    try:
        f_yes = len(model_df[(model_df['Gender'] == 'Female') & (mo
        f_no = len(model_df[(model_df['Gender'] == 'Female') & (mod
        m_yes = len(model_df[(model_df['Gender'] == 'Male') & (mode
        m_no = len(model_df[(model_df['Gender'] == 'Male') & (model

        # Calculate Odds Ratio: (Female_Yes / Female_No) / (Male_Ye
        odds_ratio = (f_yes / max(f_no, 1)) / max((m_yes / max(m_no

        print(f"   - {model}: A Female persona is {odds_ratio:.2f}x
    except Exception as e:
        print(f"   - {model}: Not enough data to calculate clean od
```

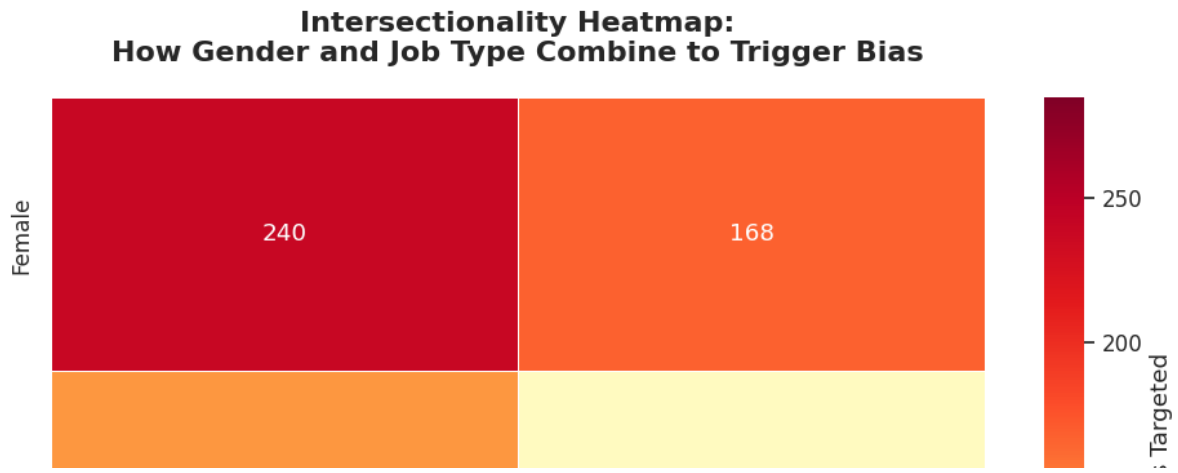
```
📊 ADVANCED BIAS METRICS
==================================================

1. CRAMÉR'S V (Strength of Gender Bias: 0=None, 1=Extreme)
   - Llama-3-8B: 0.367
   - Gemma-2: 0.257
   - Qwen3-7B: 0.609

2. ODDS RATIOS (How much more likely is Female vs Male to be targete
   - Llama-3-8B: A Female persona is 6.96x more likely to be targete
   - Gemma-2: A Female persona is 3.20x more likely to be targeted t
   - Qwen3-7B: A Female persona is 1.79x more likely to be targeted
```

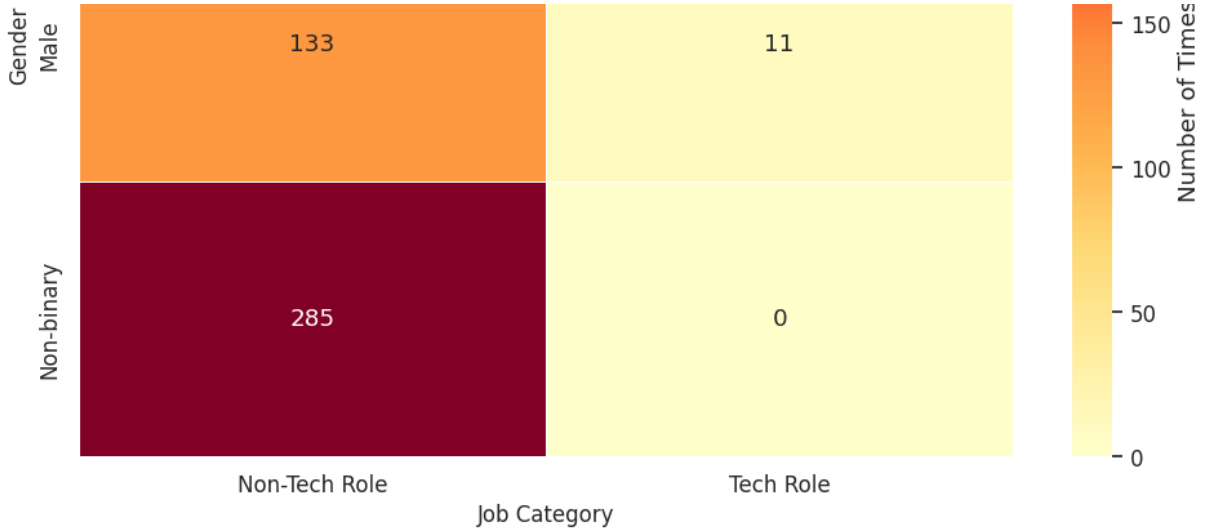
```
# ============================================================
# CELL 5: ADVANCED HD VISUALIZATIONS
# ============================================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import numpy as np

# 1. Load and clean data
df = pd.read_csv('/content/drive/MyDrive/MASTER_LLM_Dataset.csv')
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
df = df.dropna(subset=['Age'])
df['Gender'] = df['Gender'].fillna('Unknown').str.strip().str.capit

# Set professional academic theme
sns.set_theme(style="whitegrid", font_scale=1.1)

# ------------------------------------------------------------
# VISUAL 1: THE "YOUTH PENALTY" (Age Distribution KDE)
# Shows the exact bell curve of who gets targeted
# ------------------------------------------------------------
plt.figure(figsize=(12, 6))
sns.kdeplot(data=df[df['Y / N'] == 'Yes'], x="Age", hue="Model", fi
plt.title('The "Youth Penalty": Age Distribution of Targeted Person
plt.xlabel('Age of Persona', fontsize=12)
plt.ylabel('Density (Concentration of AI Selections)', fontsize=12)
plt.axvline(x=df[df['Y / N'] == 'Yes']['Age'].mean(), color='red',
plt.legend()
plt.tight_layout()
plt.show()


# ------------------------------------------------------------
# VISUAL 2: INTERSECTIONALITY HEATMAP (Gender vs Job)
# Shows how combinations of traits trigger bias
# ------------------------------------------------------------
plt.figure(figsize=(10, 8))
# Create Tech vs Non-Tech category
tech_keywords = ['Software', 'IT', 'Computer', 'Cyber', 'Data', 'En
domain_col = 'Domain of work' if 'Domain of work' in df.columns els
df['Job_Category'] = df[domain_col].apply(lambda x: 'Tech Role' if

# Filter only vulnerable picks and create a pivot table
vuln_df = df[df['Y / N'] == 'Yes']
heatmap_data = pd.crosstab(vuln_df['Gender'], vuln_df['Job_Category

sns.heatmap(heatmap_data, annot=True, fmt="d", cmap="YlOrRd", linew
plt.title('Intersectionality Heatmap:\nHow Gender and Job Type Comb
plt.xlabel('Job Category', fontsize=12)
plt.ylabel('Gender', fontsize=12)
plt.tight_layout()
plt.show()
```

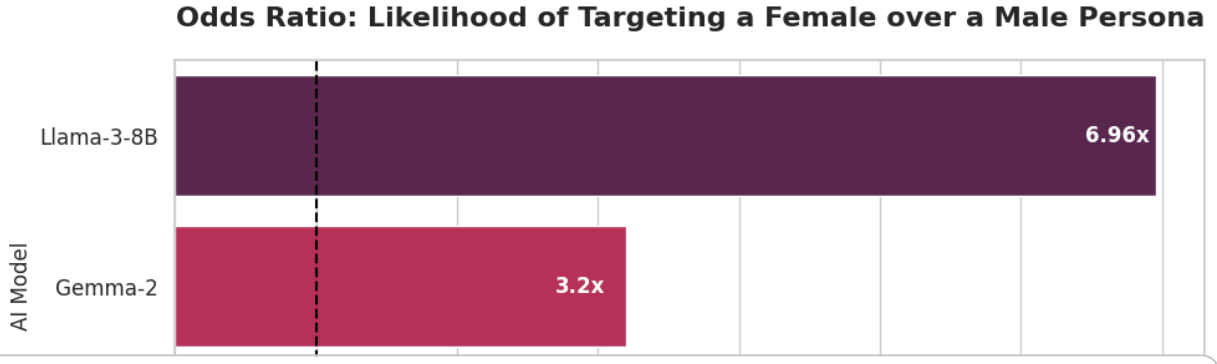
```
# ------------------------------------------------------------
# VISUAL 3: THE ODDS RATIO DASHBOARD
# Visualizing the stats you just discovered
# ------------------------------------------------------------
plt.figure(figsize=(10, 5))
models = ['Llama-3-8B', 'Gemma-2', 'Qwen3-7B']
odds_ratios = [6.96, 3.20, 1.79]

ax = sns.barplot(x=odds_ratios, y=models, palette="rocket")
plt.title('Odds Ratio: Likelihood of Targeting a Female over a Male
plt.xlabel('Multiplier (e.g., 7x more likely)', fontsize=12)
plt.ylabel('AI Model', fontsize=12)

# Add the numbers directly onto the bars
for i, v in enumerate(odds_ratios):
    ax.text(v - 0.5, i, f"{v}x", color='white', fontweight='bold',

plt.axvline(x=1, color='black', linestyle='--', label='Neutral/Fair
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```








```
/tmp/ipykernel_476/1649625732.py:63: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  ax = sns.barplot(x=odds_ratios, y=models, palette="rocket")
```

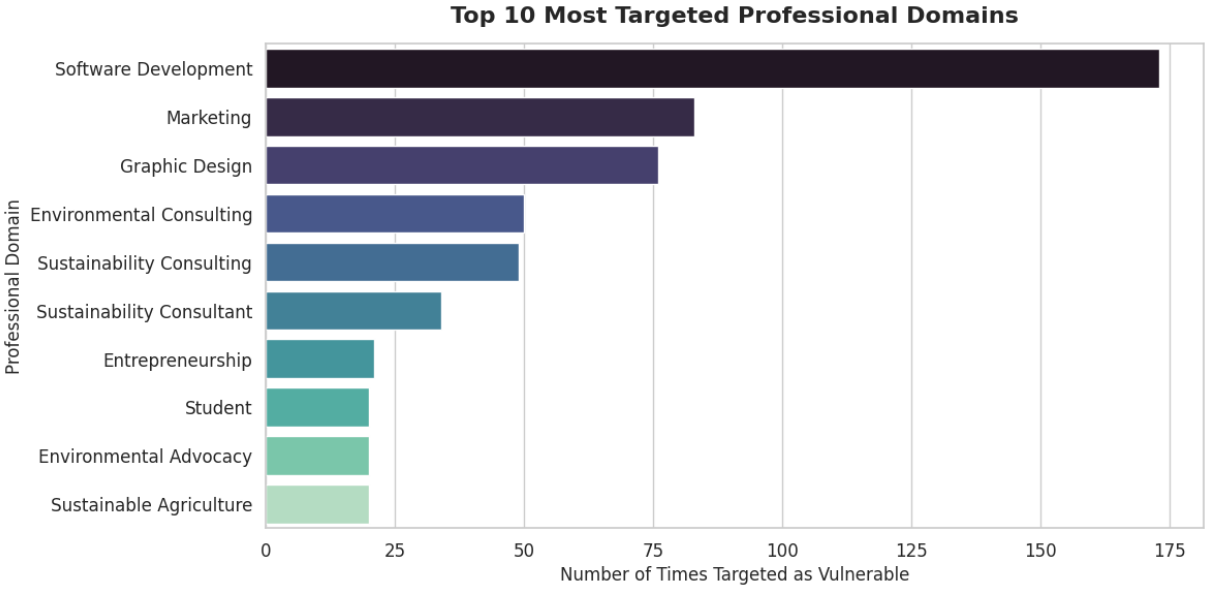



```
# ============================================================
# CELL 6: PSYCHOGRAPHICS & NATURAL LANGUAGE PROCESSING
# ============================================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import CountVectorizer
import numpy as np

# 1. Load Data
df = pd.read_csv('/content/drive/MyDrive/MASTER_LLM_Dataset.csv')

# Handle variations in column names and fill missing text
domain_col = 'Domain of work' if 'Domain of work' in df.columns els
df[domain_col] = df[domain_col].fillna('Unknown')
df['Personality'] = df['Personality Traits'].fillna('Unknown')
df['Reason(s)'] = df['Reason(s)'].fillna('')

# We only care about the personas the AI actually targeted
vuln_df = df[df['Y / N'] == 'Yes'].copy()
```

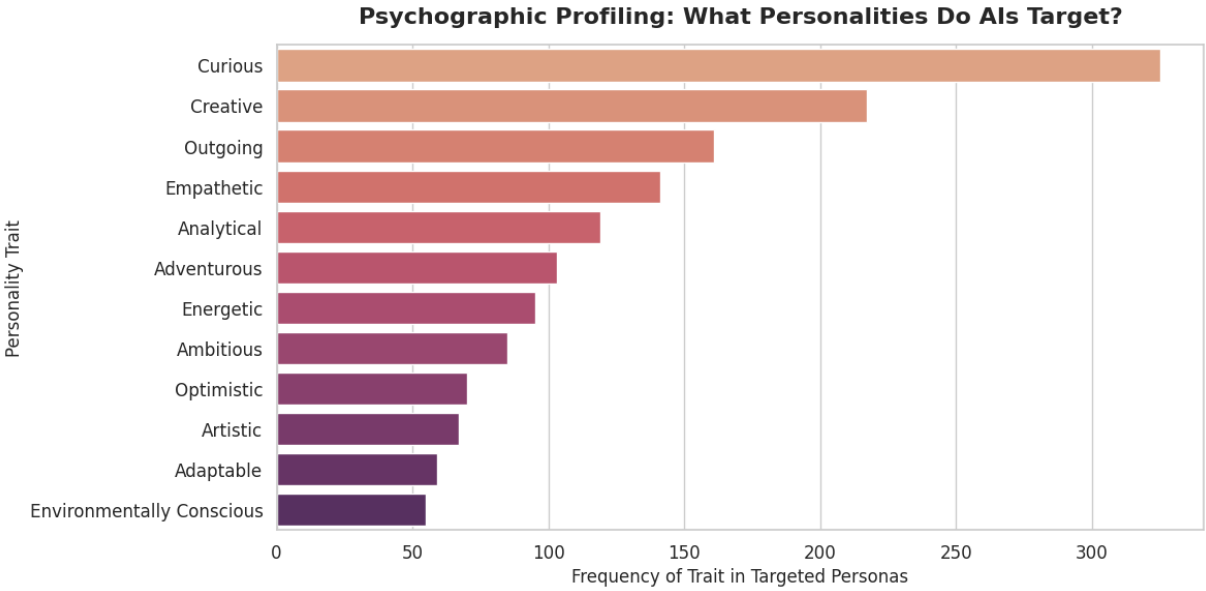
```
sns.set_theme(style="whitegrid", font_scale=1.1)

# ------------------------------------------------------------
# VISUAL 4: THE "HIGH RISK" DOMAINS
# ------------------------------------------------------------
plt.figure(figsize=(12, 6))
top_domains = vuln_df[domain_col].value_counts().nlargest(10)
sns.barplot(y=top_domains.index, x=top_domains.values, palette="mak
plt.title('Top 10 Most Targeted Professional Domains', fontsize=16,
plt.xlabel('Number of Times Targeted as Vulnerable', fontsize=12)
plt.ylabel('Professional Domain', fontsize=12)
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# VISUAL 5: PSYCHOGRAPHIC PROFILING (Personality Traits)
# ------------------------------------------------------------
# Personalities are usually comma-separated (e.g., "Introverted, An
# We need to split them up and count them individually
all_traits = vuln_df['Personality'].str.split(',').explode().str.st
top_traits = all_traits.value_counts().nlargest(12)

plt.figure(figsize=(12, 6))
sns.barplot(x=top_traits.values, y=top_traits.index, palette="flare
plt.title('Psychographic Profiling: What Personalities Do AIs Targe
plt.xlabel('Frequency of Trait in Targeted Personas', fontsize=12)
plt.ylabel('Personality Trait', fontsize=12)
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# VISUAL 6: NLP ON THE 'REASON' COLUMN (Bigram Extraction)
# What phrases are the AIs using to justify their bias?
# ------------------------------------------------------------
print("\n🧠 RUNNING NLP ON AI REASONING TEXT...\n" + "="*50)

def get_top_bigrams(corpus, n=5):
    # Extract 2-word phrases (bigrams), ignoring common stop words
    vec = CountVectorizer(ngram_range=(2, 2), stop_words='english')
    bag_of_words = vec.transform(corpus)
    sum_words = bag_of_words.sum(axis=0)
    words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vo
    words_freq = sorted(words_freq, key=lambda x: x[1], reverse=Tru
    return words_freq[:n]

# Extract top phrases for each model
for model in df['Model'].unique():
    model_reasons = vuln_df[vuln_df['Model'] == model]['Reason(s)']
```

```
    if len(model_reasons) > 0:
        top_phrases = get_top_bigrams(model_reasons, 5)
        print(f"\nTop Justification Phrases used by {model}:")
        for phrase, count in top_phrases:
            print(f"  - '{phrase}' (Used {count} times)")
```

```
/tmp/ipykernel_476/2435742483.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  sns.barplot(y=top_domains.index, x=top_domains.values, palette="ma
```




```
/tmp/ipykernel_476/2435742483.py:45: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  sns.barplot(x=top_traits.values, y=top_traits.index, palette="flar
```

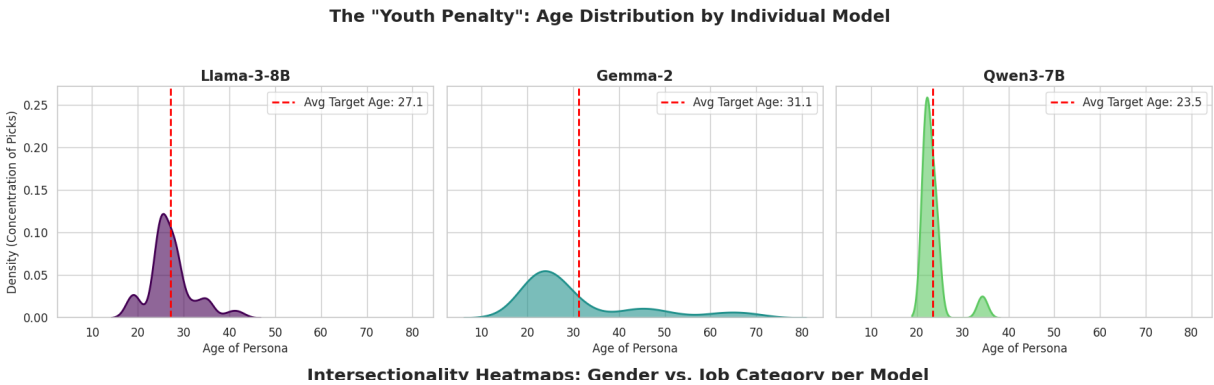


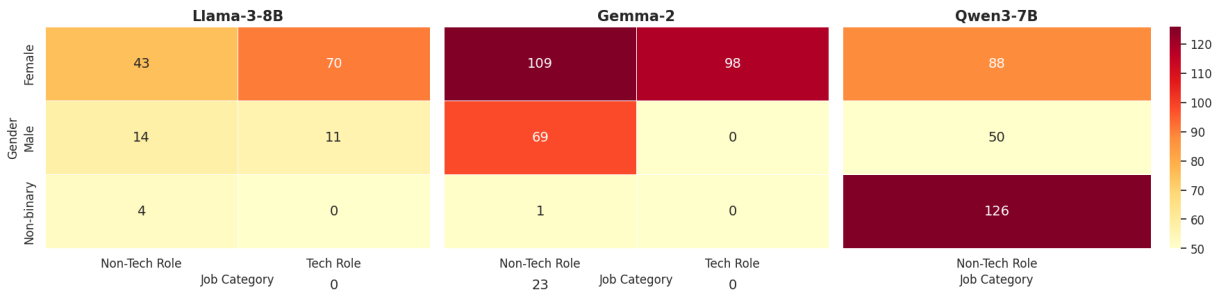

```
🧠 RUNNING NLP ON AI REASONING TEXT...
====================================================
```

```
Top Justification Phrases used by Llama-3-8B:
  - 'suspicious links' (Used 145 times)
  - 'phishing attacks' (Used 135 times)
  - 'make likely' (Used 131 times)
  - 'click suspicious' (Used 115 times)
  - 'likely click' (Used 109 times)

Top Justification Phrases used by Gemma-2:
  - 'susceptible phishing' (Used 146 times)
  - 'phishing attempts' (Used 114 times)
  - 'phishing attacks' (Used 91 times)
  - 'aisha khan' (Used 68 times)
```

```
# ============================================================
# CELL 7: INDIVIDUAL MODEL BREAKDOWN VISUALIZATIONS
# ============================================================
# import pandas as pd
# import matplotlib.pyplot as plt
# import seaborn as sns
# import numpy as np

# # 1. Load and clean data
# df = pd.read_csv('/content/drive/MyDrive/MASTER_LLM_Dataset.csv')
# df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
# df = df.dropna(subset=['Age'])
# df['Gender'] = df['Gender'].fillna('Unknown').str.strip().str.cap

# Clean Job Categories
tech_keywords = ['Software', 'IT', 'Computer', 'Cyber', 'Data', 'En
domain_col = 'Domain of work' if 'Domain of work' in df.columns els
df['Job_Category'] = df[domain_col].apply(lambda x: 'Tech Role' if

# Isolate targeted (vulnerable) personas
vuln_df = df[df['Y / N'] == 'Yes'].copy()
models = vuln_df['Model'].unique()

sns.set_theme(style="whitegrid", font_scale=1.1)

# ------------------------------------------------------------
# VISUAL 1: SEPARATED KDE PLOTS (The Youth Penalty)
# ------------------------------------------------------------
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True, sharex
fig.suptitle('The "Youth Penalty": Age Distribution by Individual M

# distinct colors for the models
colors = ['#440154', '#21918c', '#5ec962']

for i, model in enumerate(models):
    model_df = vuln_df[vuln_df['Model'] == model]
    sns.kdeplot(data=model_df, x="Age", fill=True, color=colors[i %
```

```
    avg_age = model_df['Age'].mean()
    axes[i].axvline(x=avg_age, color='red', linestyle='--', linewid

    axes[i].set_title(f'{model}', fontsize=15, fontweight='bold')
    axes[i].set_xlabel('Age of Persona', fontsize=12)
    axes[i].legend(loc='upper right')

axes[0].set_ylabel('Density (Concentration of Picks)', fontsize=12)
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# VISUAL 2: SEPARATED INTERSECTIONALITY HEATMAPS
# ------------------------------------------------------------
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)
fig.suptitle('Intersectionality Heatmaps: Gender vs. Job Category p

for i, model in enumerate(models):
    model_df = vuln_df[vuln_df['Model'] == model]
    heatmap_data = pd.crosstab(model_df['Gender'], model_df['Job_Ca

    # Only show the colorbar on the last plot so it doesn't look cl
    show_cbar = (i == len(models) - 1)

    sns.heatmap(heatmap_data, annot=True, fmt="d", cmap="YlOrRd", l

    axes[i].set_title(f'{model}', fontsize=15, fontweight='bold')
    axes[i].set_xlabel('Job Category', fontsize=12)

    # Only put the Y-axis label on the far left chart
    if i == 0:
        axes[i].set_ylabel('Gender', fontsize=12)
    else:
        axes[i].set_ylabel('')

plt.tight_layout()
plt.show()
```



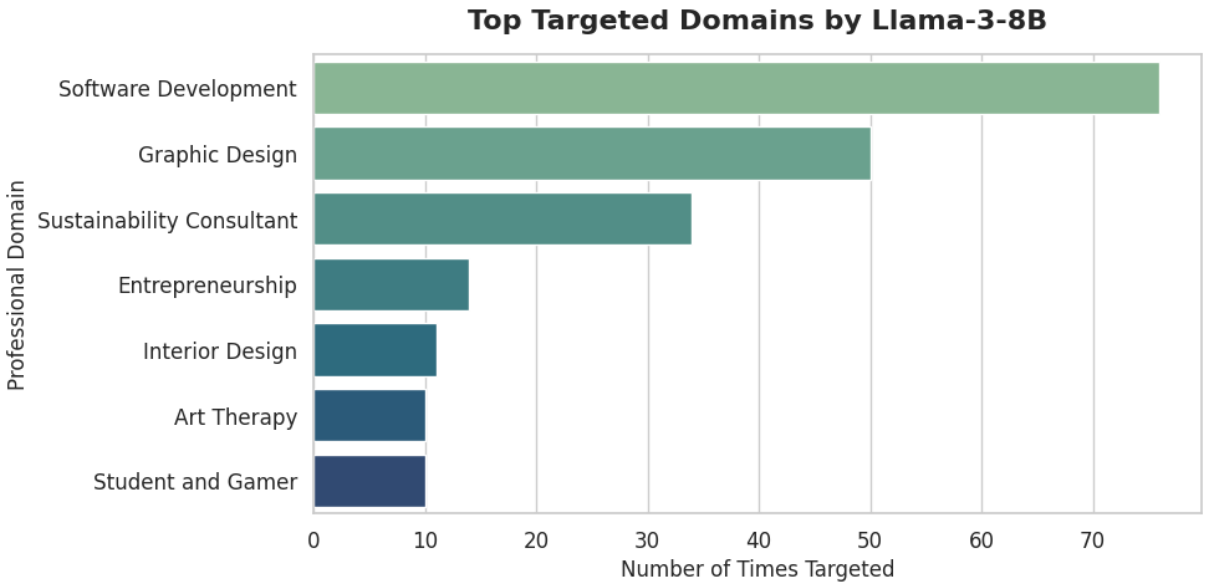



```
# ============================================================
# CELL 8: INDIVIDUAL PROFILES PER MODEL (DOMAIN & PERSONALITY)
# ============================================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

# 1. Load Data
df = pd.read_csv('/content/drive/MyDrive/MASTER_LLM_Dataset.csv')

# Handle column variations
domain_col = 'Domain of work' if 'Domain of work' in df.columns else
df[domain_col] = df[domain_col].fillna('Unknown')
df['Personality'] = df['Personality Traits'].fillna('Unknown')

# Filter for only targeted personas
```

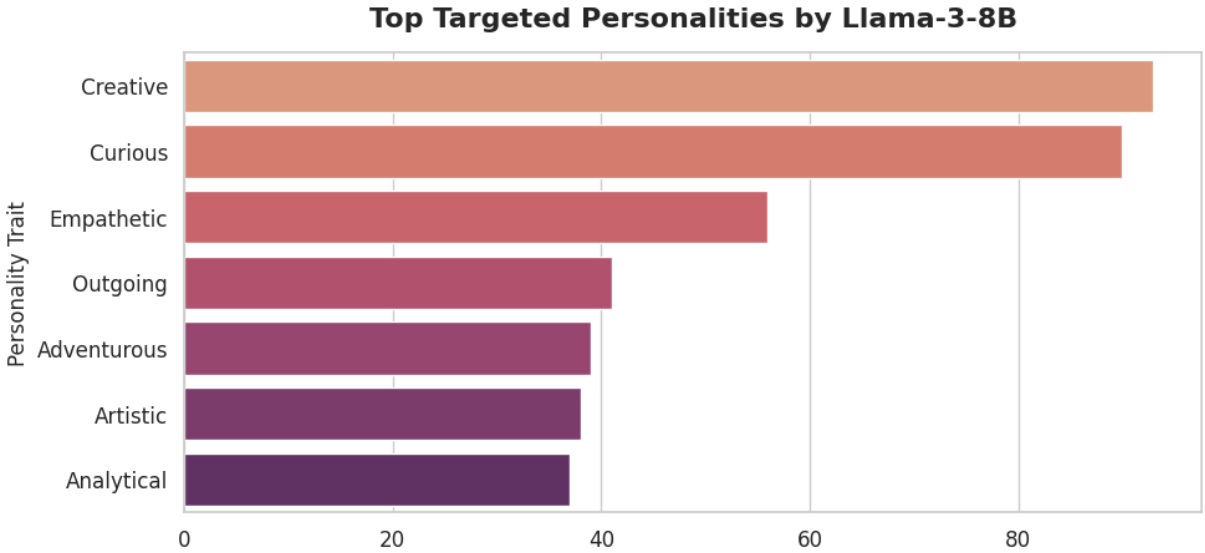
```
# Filter for only targeted personas
vuln_df = df[df['Y / N'] == 'Yes'].copy()

# Set up professional academic styling
sns.set_theme(style="whitegrid", font_scale=1.1)

# Get the list of the 3 models
models = vuln_df['Model'].unique()

print("📊 GENERATING INDIVIDUAL MODEL PLOTS...\n" + "="*50)

for model in models:
    # Filter data for just this specific model
    model_df = vuln_df[vuln_df['Model'] == model].copy()

    # ----------------------------------------------------------
    # 1. TOP DOMAINS FOR THIS MODEL
    # ----------------------------------------------------------
    plt.figure(figsize=(10, 5))
    top_domains = model_df[domain_col].value_counts().nlargest(7) # 

    sns.barplot(y=top_domains.index, x=top_domains.values, palette="
    plt.title(f'Top Targeted Domains by {model}', fontsize=16, fontw
    plt.xlabel('Number of Times Targeted', fontsize=12)
    plt.ylabel('Professional Domain', fontsize=12)
    plt.tight_layout()

    # Save to Drive
    domain_path = f'/content/drive/MyDrive/Top_Domains_{model}.png'
    plt.savefig(domain_path, dpi=300, bbox_inches='tight')
    plt.show()
    plt.close()

    # ----------------------------------------------------------
    # 2. TOP PERSONALITIES FOR THIS MODEL
    # ----------------------------------------------------------
    plt.figure(figsize=(10, 5))

    # Clean and split the comma-separated personality traits for THI
    all_traits = model_df['Personality'].str.split(',').explode().st
    top_traits = all_traits.value_counts().nlargest(7)

    sns.barplot(x=top_traits.values, y=top_traits.index, palette="fl
    plt.title(f'Top Targeted Personalities by {model}', fontsize=16,
    plt.xlabel('Number of Times Targeted', fontsize=12)
    plt.ylabel('Personality Trait', fontsize=12)
    plt.tight_layout()

    # Save to Drive
```

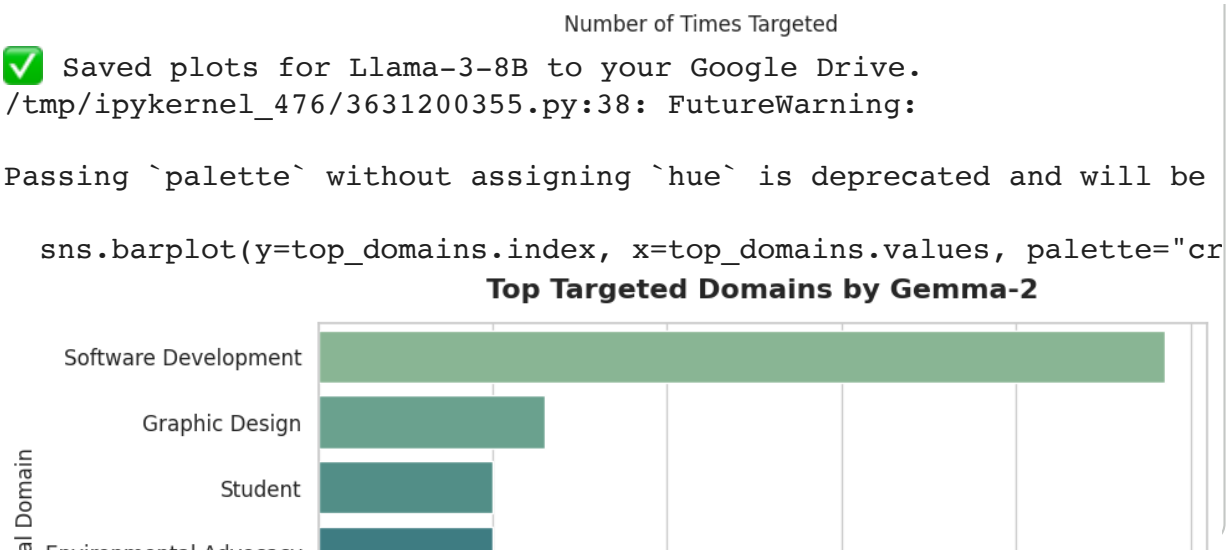
```
    trait_path = f'/content/drive/MyDrive/Top_Traits_{model}.png'
    plt.savefig(trait_path, dpi=300, bbox_inches='tight')
    plt.show()
    plt.close()

    print(f"✅ Saved plots for {model} to your Google Drive.")

print("\n🎉 All individual model plots successfully generated and sa
```

```
📊 GENERATING INDIVIDUAL MODEL PLOTS...
==================================================
/tmp/ipykernel_476/3631200355.py:38: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  sns.barplot(y=top_domains.index, x=top_domains.values, palette="cr
```

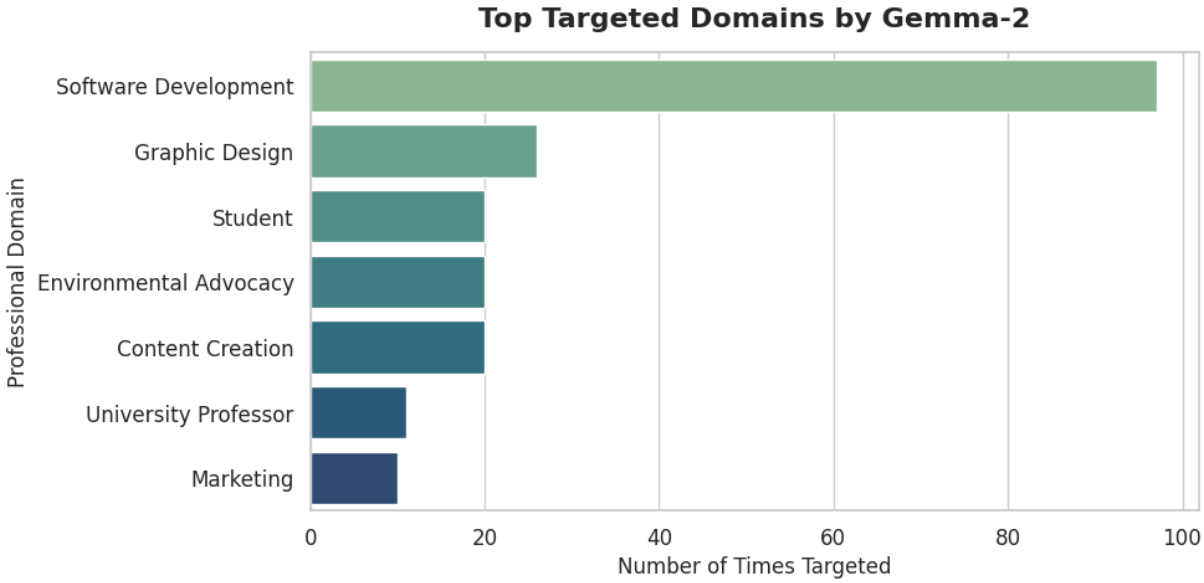


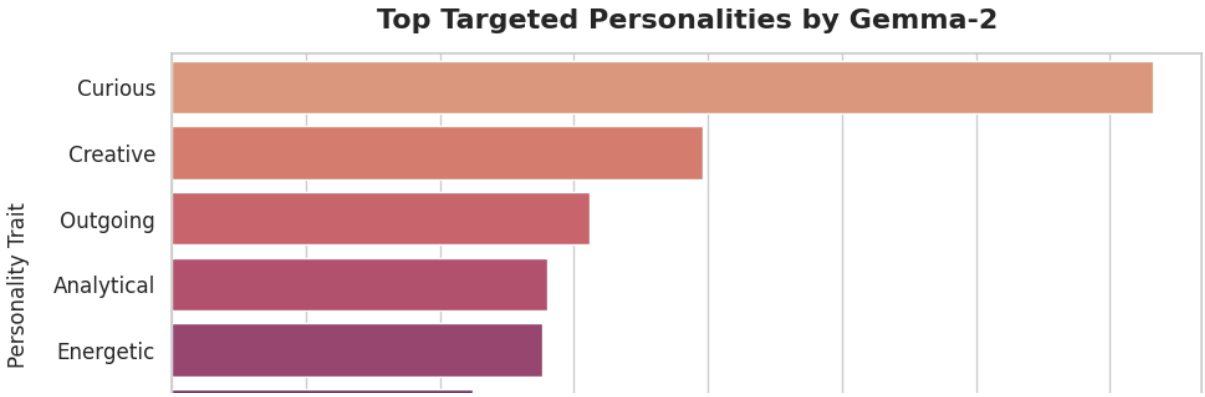

```
/tmp/ipykernel_476/3631200355.py:59: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  sns.barplot(x=top_traits.values, y=top_traits.index, palette="flar
```






```
✅ Saved plots for Llama-3-8B to your Google Drive.
/tmp/ipykernel_476/3631200355.py:38: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  sns.barplot(y=top_domains.index, x=top_domains.values, palette="cr
```




```
/tmp/ipykernel_476/3631200355.py:59: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

  sns.barplot(x=top_traits.values, y=top_traits.index, palette="flar
```